Finite Markov Decision Processes

Sharanya Bhatia

March 1, 2026

Contents

1 Introduction **2**

2 The Agent–Environment Interface **2**

3 Goals and Rewards **2**

4 Returns and Episodes **2**

4.1 Return 2

4.2 Episodic Tasks 3

4.3 Continuing Tasks 3

5 Unified Notation for Episodic and Continuing Tasks **3**

6 Policies and Value Functions **3**

6.1 State-Value Function 3

6.2 Action-Value Function 4

6.3 Bellman Expectation Equation 4

7 Optimal Policies and Optimal Value Functions **5**

7.1 Value Iteration 5

8 Worked Numerical Example **5**

9 Optimality and Approximation **6**

10 Conclusion **7**

1 Introduction

Reinforcement Learning (RL) studies how an agent can learn to make decisions by interacting with an environment. The main mathematical framework behind RL is the **Markov Decision Process (MDP)**. MDPs provide a way to model sequential decision-making problems where outcomes are partly random and partly under the control of an agent.

2 The Agent–Environment Interface

At each discrete time step t:

- The agent observes state $S_t \in S$
- It selects an action $A_t \in A$
- The environment returns reward R_{t+1}
- The system transitions to state S_{t+1}

This interaction is controlled by transition probabilities:

$$P(s', r|s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$$

The defining property is the **Markov property**:

$$\Pr(S_{t+1}|S_t, A_t, \dots) = \Pr(S_{t+1}|S_t, A_t)$$

Thus, the current state contains all necessary information about the past.

3 Goals and Rewards

The goal of the agent is not to maximize immediate reward, but to maximize the **expected cumulative reward**. This idea is formalized through the *return*.

The **reward hypothesis** states:

All goals can be described as the maximization of expected cumulative reward.

The reward function $R(s, a)$ defines the objective of the system.

4 Returns and Episodes

4.1 Return

The discounted return is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $0 \leq \gamma \leq 1$ is the discount factor.

4.2 Episodic Tasks

In episodic tasks, interaction terminates after some time T:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

A terminal state is explained as an absorbing state with zero future reward.

4.3 Continuing Tasks

In continuing tasks, interaction does not terminate. Discounting ($\gamma < 1$) ensures the return remains finite.

5 Unified Notation for Episodic and Continuing Tasks

Both episodic and continuing tasks can be expressed using:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

For episodic problems, after terminal time T, rewards are zero. Thus, the same mathematical expression applies to both settings.

6 Policies and Value Functions

A policy π specifies the agent's behavior:

$$\pi(a|s) = \Pr(A_t = a \mid S_t = s)$$

6.1 State-Value Function

The state-value function measures how good it is for the agent to be in a particular state while following a policy π. Formally,

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

it averages over both:

- The randomness in the environment (state transitions), and
- The randomness in the policy (if π is stochastic).

$$G_t = R_{t+1} + \gamma G_{t+1},$$

we can interpret $V^\pi(s)$ as:

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s].$$

Thus, the value of a state consists of two parts:

- Immediate reward,
- Discounted value of the next state.

6.2 Action-Value Function

While $V^\pi(s)$ evaluates states, the action-value function evaluates state–action pairs:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Here, we assume the agent first takes action a in state s, and then continues following policy π afterward.

Using the same recursive reasoning:

$$Q^\pi(s, a) = \mathbb{E}\left[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = a\right].$$

$$Q^\pi(s, a) = \sum_{s'} P(s'|s, a)\left[R(s, a) + \gamma V^\pi(s')\right].$$

The relationship between V^π and Q^π is:

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a).$$

This equation clearly shows that the value of a state is simply the weighted average of the action-values, weighted by the policy's action probabilities.

6.3 Bellman Expectation Equation

$$G_t = R_{t+1} + \gamma G_{t+1}$$

Taking expectation:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a) \left[R(s,a) + \gamma V^{\pi}(s') \right]$$

This equation expresses a fundamental recursive relationship.

7 Optimal Policies and Optimal Value Functions

The optimal value function is defined as:

$$V^{*}(s) = \max_{\pi} V^{\pi}(s)$$

It satisfies the Bellman Optimality Equation:

$$V^{*}(s) = \max_{a} \sum_{s'} P(s'|s,a) \left[R(s,a) + \gamma V^{*}(s') \right]$$

An optimal policy can be defined as:

$$\pi^{*}(s) = \arg\max_{a} \sum_{s'} P(s'|s,a) \left[R(s,a) + \gamma V^{*}(s') \right]$$

7.1 Value Iteration

$$V_{k+1}(s) = \max_{a} \sum_{s'} P(s'|s,a) \left[R(s,a) + \gamma V_{k}(s') \right]$$

This update converges because the Bellman operator is a contraction:

$$\|TV - TW\|_{\infty} \leq \gamma \|V - W\|_{\infty}$$

8 Worked Numerical Example

Consider an MDP with two states $\{s_1, s_2\}$ and one action per state.

Transition:

$$P(s_1|s_1) = 0.5, \quad P(s_2|s_1) = 0.5$$

$$P(s_2|s_2) = 1$$

Rewards:

$$R(s_1) = 5, \quad R(s_2) = 2$$

Let $\gamma = 0.9$.

Bellman equations:

$$V(s_1) = 5 + 0.9(0.5V(s_1) + 0.5V(s_2))$$

$$V(s_2) = 2 + 0.9V(s_2)$$

Solving:

$$V(s_2) = \frac{2}{1 - 0.9} = 20$$

Substituting:

$$V(s_1) = 5 + 0.9(0.5V(s_1) + 10)$$

$$V(s_1) = 5 + 0.45V(s_1) + 9$$

$$0.55V(s_1) = 14$$

$$V(s_1) = 25.45$$

This illustrates how value functions are computed.

9 Optimality and Approximation

In small finite MDPs, value functions can be computed exactly using value iteration or by solving systems of linear equations. But, in real-world problems, the state space may be extremely large or even continuous. In such cases, storing a separate value for every state is important.

To do this, we approximate the value function using parameterized models:

$$V(s) \approx V_\theta(s) = \theta^T \phi(s).$$

Here:

- $\phi(s)$ is a feature vector representing state s,
- θ is a parameter vector,
- $\theta^T \phi(s)$ denotes their inner product.

Instead of solving the Bellman equation exactly, we aim to make $V_\theta(s)$ as close as possible to satisfying it. One natural objective is to minimize the **Bellman error**:

$$\min_{\theta} \|V_\theta - TV_\theta\|^2,$$

where T is the Bellman operator defined as:

$$(TV)(s) = \max_a \sum_{s'} P(s'|s,a)\left[R(s,a) + \gamma V(s')\right].$$

- V_θ is our current estimate of the value function.
- TV_θ is the value obtained after applying one Bellman update.
- Ideally, we would like the value function to satisfy the Bellman equation exactly, i.e., $V = TV$.
- In large or continuous problems, solving this equation exactly is usually not feasible.
- Therefore, we aim to make V_θ as close as possible to TV_θ.

10 Conclusion

Finite Markov Decision Processes (MDPs) offer a structured way to understand decision-making over time. Starting from agent–environment interaction, ideas like return, value functions, and the Bellman equations. Their repetitive nature leads to optimal solutions, and even when problems grow large, approximation methods keep them practical.